Module: 2 Docker file Construction

Assignment/Task: Dockerfile understadings

Docker File Instruc7ons:

- FROM

- Used for declare the base image like python:3.12, Node.js:1.3 etc…

- WORKDIR

- Used for seIng the working directory of the project.

- COPY

- Used for copy the files and directories

- ADD

- Add local or remote files and directories.

- RUN

- Used to Execute commands.

- ENV

- Set environment variables.

- ARG

- Use build-Vme variables.

- EXPOSE

- Describe which ports your applicaVon is listening on.

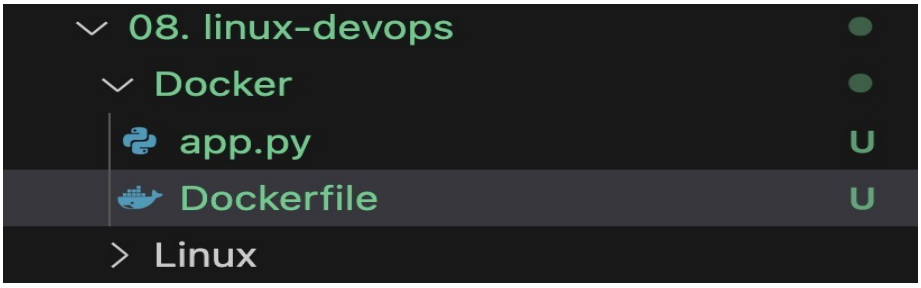
- CMD

- Specify default commands.

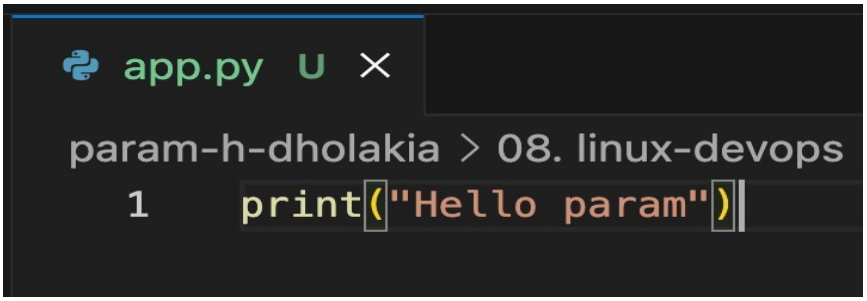
- ENTRYPOINT

- Specify default executable.

File Structure:



app.py file:



DockerFile:

```
FROM python:3.12-slim


WORKDIR /app

COPY app.py .
EXPOSE 80


ENTRYPOINT ["python"]
CMD ["app.py"]
```

Build Logs: **docker build -t paramdocker-demo1**

```
param@Params-MacBook-Air Docker % docker build -t param-docker-demo1 .
[+] Building 1.2s (9/9) FINISHED                                                docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                0.0s
 => => transferring dockerfile: 225B                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                 0.9s
 => [internal] load .dockerignore                                                                   0.0s
 => => transferring context: 2B                                                                     0.0s
 => CACHED [1/4] FROM docker.io/library/python:3.12-slim@sha256:9e01bf1ae5db7649a236da7be1e94ffbbbdd7a93f86  0.0s
 => => resolve docker.io/library/python:3.12-slim@sha256:9e01bf1ae5db7649a236da7be1e94ffbbbdd7a93f867dd0d8d  0.0s
 => [internal] load build context                                                                   0.0s
 => => transferring context: 53B                                                                    0.0s
 => [2/4] WORKDIR /app                                                                              0.0s
 => [3/4] RUN echo "build no is: 1"                                                                 0.1s
 => [4/4] COPY app.py .                                                                             0.0s
 => exporting to image                                                                              0.1s
 => => exporting layers                                                                             0.0s
 => => exporting manifest sha256:ae08ea198d71641e2a4213e7049b8b18ca01de93e652114a0ff8ec4f2c23f544   0.0s
 => => exporting config sha256:448d54cd11c2763e7b3314404b03f571553b7ab7374eee11cd63e7d3e30f3e72      0.0s
 => => exporting attestation manifest sha256:6e7d97d6b28d3a65d505ab49ab48d489d896f6dc64f60bbe3c93e835033861  0.0s
 => => exporting manifest list sha256:2ff50734935c2f75c214df1f401b9905c0e3f681fcabf0c6b0470f55d4be92d7  0.0s
 => => naming to docker.io/library/param-docker-demo1:latest                                        0.0s
 => => unpacking to docker.io/library/param-docker-demo1:latest                                     0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/3msphfhthv56ufyzdl324s67b
param@Params-MacBook-Air Docker %
```

Run the container: **docker run param-dockerdemo1**

```
param@Params-MacBook-Air Docker % docker run param-docker-demo1

Hello param
param@Params-MacBook-Air Docker %
```

- **How to select an appropriate base image** By using a lightweight and stable Python image: FROM python:3.12-slim

---

- **How to set the working directory inside the container**
  Set a dedicated applicaYon directory: WORKDIR /app

- **How to copy applica6on files into the container**

Use the COPY instrucYon to transfer your Python file into the container: COPY app.py .

- **How to define build-6me arguments (op6onal)** Using ARG to define build variables:

ARG BUILD_ID=1
RUN echo "build no is: ${BUILD_ID}"

- **How to define environment variables** Using ENV instrucYon:

ENV APP_NAME="web status checker"

- **How to expose an applica6on port** By using the EXPOSE instrucYon: EXPOSE 80

- **How to define the container startup command** Using ENTRYPOINT and CMD instrucYons:

ENTRYPOINT ["python"] CMD
["app.py"]